

Infra

Morning Flower and Evening Oath

Python、C++ 进阶、Shell 与赛场调试

目录

第一章 Python	1
1.1 精确整数、分数与小数	1
1.2 枚举、容器与记忆化	2
1.3 随机造数与打表找规律	3
1.4 输入输出与容易踩的坑	4
第二章 C++ 进阶	5
2.1 优化选项与局部 pragma	5
2.2 浮点优化会改变答案	6
2.3 O2 基线下怎样测	6
2.4 先减少复制、分配和跨步访问	7
2.5 复用缓冲与输入输出	8
2.6 先检查自动向量化	9
2.7 AVX2 类型与常用指令	9
2.8 AVX2 批量加法与条件计数	10
2.9 重排与 AVX-512 掩码尾部	11
2.10 原生 Linux 实测	11
2.11 编译警告与检查配置	12
2.12 读懂第一条错误	13
2.13 值、引用与调用边界	14
2.14 std::move: 允许移动, 不是立即移动	14
2.15 vector、array、deque 与字符串	14
2.16 set、map 与哈希表	14
2.17 queue、stack 与优先队列	15
2.18 排序、查找与区间算法	15
2.19 pair、tuple、optional 与回调	15
2.20 数值累加与前缀和	16
2.21 complex 与 FFT	16
2.22 C++20 位操作与零边界	16
2.23 计时与卡时	17
2.24 随机引擎、分布与抽样	17
2.25 对拍用造数数据函数	18
2.26 生成器入口	19
2.27 整数键防卡哈希	20
2.28 GNU 扩展与查询范围	20
2.29 Boost 任意精度整数	20
2.30 __int128 的十进制读写	21
2.31 strict 与 GNU 模式的差异	22
2.32 GNU 位函数与 bitset 遍历	23
第三章 Shell 脚本与常用命令	24
3.1 编译并运行	24
3.2 对拍与保存反例	25
3.3 完整造数与暴力示例	26
第四章 赛场环境与调试	27
4.1 提交前的接口核对	27
4.2 对拍与性能检查	27
4.3 分册跳转索引	27

第一章 Python

1.1 精确整数、分数与小数

需要整数答案时保留整数运算；不要先除成 float 再转回 int。

用途	最短写法与坑点
整数平方根	<code>isqrt(n)</code> ，要求非负；返回向下取整的精确结果。
组合数、最大公约数	<code>comb(n,k)</code> 、 <code>gcd(a,b)</code> 、 <code>lcm(a,b)</code> 。
模幂、模逆	<code>pow(a,e,m)</code> ；逆元用 <code>pow(a,-1,m)</code> ，不互质抛 <code>ValueError</code> 。
整数除法	// 向下取整，C++ 向零截断；Python 余数与除数同号或为零。
有理数	<code>Fraction(a,b)</code> 自动约分；十进制输入用字符串，避免先变 float。
高精度小数	<code>Decimal("0.1")</code> ；context 的 <code>prec</code> 是有效数字数，并非小数位数。

```
1 from math import comb, gcd, isqrt, lcm
2 from fractions import Fraction
3 from decimal import Decimal, localcontext
4
5 n = 10**100
6 assert isqrt(n - 1) == 10**50 - 1
7 assert comb(100, 50) == 100891344545564193334812497256
8 assert gcd(12, 18) == 6
9 assert lcm(12, 18) == 36
10 assert pow(3, 100, 7) == 4
11 assert pow(3, -1, 7) == 5
12 assert divmod(-7, 3) == (-3, 2)
13 assert (-7) // 3 == -3
14 assert -(7 // 3) == -2
15
16 a = Fraction(1, 3) + Fraction(1, 6)
17 assert (a.numerator, a.denominator) == (1, 2)
18 assert Fraction("0.1") == Fraction(1, 10)
19 assert Fraction(0.1) != Fraction(1, 10)
20 with localcontext() as ctx:
21     ctx.prec = 60
22     x = Decimal(2).sqrt()
23     assert abs(x * x - 2) < Decimal("1e-58")
24     assert Decimal("0.1") * 3 == Decimal("0.3")
25 print("numbers: pass")
```

`Fraction` 的分子分母可能快速膨胀；`Decimal` 仍有舍入，不能当作任意精确实数。

1.2 枚举、容器与记忆化

`combinations` 枚举选下标的组合, `permutations` 枚举排列, `product` 枚举笛卡尔积。输入有重复值时不自动去重。它们产生迭代器; 大规模枚举直接遍历, 别先转成 `list`。

```

1 from collections import Counter, defaultdict, deque
2 from itertools import accumulate, combinations, permutations, product
3 from heapq import heapify, heappop, heappush
4 from bisect import bisect_left, bisect_right
5 from functools import cache
6
7 assert list(combinations(range(4), 2))[-1] == (2, 3)
8 assert len(list(permutations(range(4)))) == 24
9 assert len(list(product(range(2), repeat=5))) == 32
10 assert list(accumulate([2, -1, 4], initial=0)) == [0, 2, 1, 5]
11 assert Counter("ababa")["a"] == 3
12 g = defaultdict(list)
13 g[2].append(3)
14 assert g[2] == [3]
15 q = deque([1, 2])
16 q.appendleft(0)
17 assert q.popleft() == 0
18 h = [5, 2, 7]
19 heapify(h)
20 heappush(h, 1)
21 assert heappop(h) == 1
22 a = [1, 2, 2, 4]
23 assert bisect_right(a, 2) - bisect_left(a, 2) == 2
24
25 @cache
26 def paths(r, c):
27     if r == 0 or c == 0:
28         return 1
29     return paths(r - 1, c) + paths(r, c - 1)
30
31 assert paths(10, 10) == 184756
32 paths.cache_clear()
33 rows = [[0] * 2 for _ in range(3)]
34 rows[0][0] = 7
35 assert rows[1][0] == 0
36 print("containers: pass")

```

`heapq` 默认小根堆; 大根堆可存负值。 `bisect` 要求序列有序, 二分 $O(\log n)$, 向 `list` 中间插入仍 $O(n)$ 。 `Counter` 读不存在的键返回 0; `defaultdict` 下标读取可能插入新键。 `cache` 参数必须可哈希, 多组改变外部状态后清缓存。递归深度仍受限。

1.3 随机造数与打表找规律

固定种子复现失败输入；不同 Python 版本不承诺所有抽样函数逐项一致，保存实际输入。randint 两端包含；randrange 右端不含，第三参数是步长。sample 无放回抽取位置，输入有重复值时结果仍可能重复；shuffle 原地改序列并返回 None。

```
1 import random
2
3 rng = random.Random(2026)
4 a = rng.randint(-5, 5)
5 b = rng.randrange(2, 10, 2)
6 c = rng.choice("abc")
7 v = rng.sample(range(100), 10)
8 rng.shuffle(v)
9 assert -5 <= a <= 5
10 assert b in (2, 4, 6, 8)
11 assert c in "abc"
12 assert len(set(v)) == 10
13 rng.seed(42)
14 first = [rng.randrange(1000) for _ in range(10)]
15 rng.seed(42)
16 assert first == [rng.randrange(1000) for _ in range(10)]
17 print("random: pass")
```

下例暴力统计无相邻 1 的二进制串，检查前 11 项是否符合猜测。有限打表只能发现规律；本例递推证明为按末位 0、末尾 01 分类，另处理长度 0 和 1。

```
1 from itertools import product
2
3 # Binary strings with no adjacent ones, including the empty string.
4 counts = []
5 for n in range(11):
6     good = 0
7     for a in product(range(2), repeat=n):
8         if all(a[i - 1] + a[i] < 2 for i in range(1, n)):
9             good += 1
10    counts.append(good)
11    print(n, good)
12 assert counts[:2] == [1, 2]
13 assert all(counts[n] == counts[n - 1] + counts[n - 2]
14            for n in range(2, len(counts)))
```

1.4 输入输出与容易踩的坑

```
1 import sys
2
3 # Input: n followed by n integers, possibly across multiple lines.
4 it = iter(map(int, sys.stdin.buffer.read().split()))
5 n = next(it)
6 a = [next(it) for _ in range(n)]
7 sys.stdout.write(str(sum(a)) + "\n")
8 sys.stdout.write(" ".join(map(str, a)) + "\n")
```

`read().split()` 同时保存全部 token，大输入可改成逐行读。`bytes` 下标得到整数；读文本才得到单字符字符串。大量输出用 `join` 或分块缓冲，避免循环逐字符拼接字符串。

场景	做法
二维数组	<code>[[0]*m for _ in range(n)]</code> ; <code>[[0]*m]*n</code> 共享同一行。
复制	<code>b=a</code> 为别名; <code>a[:]</code> 只复制外层。
递归	<code>sys.setrecursionlimit(k)</code> 调整解释器限制, 不增加操作系统栈与内存。深递归须在比赛环境实测。
选择解释器	CPython 启动快、C 实现内置操作强; PyPy 的 JIT 可能加速长时间纯 Python 循环, 但有预热和额外内存。用相同最大数据分别实测。
大整数	<code>int</code> 不溢出, 但操作成本随位数增长; 避免 <code>int(a/b)</code> 做大整数除法。
十进制转换	CPython 3.11+ 可限制十进制转换位数; 只对可信比赛输入按下例关闭。

```
1 import sys
2
3 # Trusted contest input only; affects conversions, not int arithmetic.
4 sys.set_int_max_str_digits(0)
5 x = int("9" * 5000)
6 assert len(str(x)) == 5000
7 assert len(str(x + 1)) == 5001
8 print("digits: pass")
```

该限制作用于十进制等转换，不限制 `int` 本身的算术位数；二进制、十六进制转换不受同一限制。

第二章 C++ 进阶

2.1 优化选项与局部 pragma

先选合适算法和数据结构，再定位热点；以评测的 O2 配置为基线实测。优化不是纠正未定义行为的工具，调试版与提交版要分别运行。

选项	用法与代价
-O2	常用优化基线。具体启用项依 GCC 版本和目标而变。
-O3	增加循环等变换，可能增大代码；不保证比 O2 快。
-funroll-loops	尝试展开循环，减少部分循环控制开销；代码膨胀可能伤害指令缓存。
-Ofast	含 fast-math 等放宽语义的优化；浮点重排、NaN/Inf、带符号零等处理可能变化。
GCC optimize	给后续函数定义设置优化选项，不追溯先前定义。
GCC target	为后续函数选目标特性；启用指令集不等于检测运行 CPU。

下面仅在一个函数上试 O3 和循环展开；push/pop 恢复进入前的设置。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 #pragma GCC push_options
5 #pragma GCC optimize("O3", "unroll-loops")
6 uint64_t sum_values(const vector<unsigned>& a) {
7     uint64_t sum = 0;
8     for (unsigned x : a) {
9         sum += x;
10    }
11    return sum;
12 }
13 #pragma GCC pop_options
14
15 int main() {
16     vector<unsigned> a{1, 2, 3};
17     cout << sum_values(a) << '\n';
18 }
```

把 pragma 放在 #include 前，头文件内后续定义的函数、内联函数和模板也可能受影响；放在 include 后不追溯已读入的定义。头文件自己的属性或 push/pop 还可能调整作用范围。优先在头文件之后，只包住已测过的热点；不要无差别堆叠全局“火车头”。reset_options 恢复命令行的默认选项，pop_options 恢复上次 push 的设置。

x86 的 target("avx2") 允许生成 AVX2 代码，不自动判断评测 CPU 是否支持。执行 CPU 或操作系统未支持的指令可能触发非法指令退出（常见 SIGILL）。更高指令集不必然更快；目标函数与调用者特性不一致也可能导致内联编译错误。热身时确认评测 CPU、系统和编译器支持，再选择目标指令集。

2.2 浮点优化会改变答案

下例输入 10000000000000000 1。普通 double 计算先把加 1 舍入掉；允许代数重排后，编译器可以把整个表达式化为 y 。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 __attribute__((noinline))
5 double expression(double x, double y) {
6     return (x + y) - x;
7 }
8
9 int main() {
10     double x;
11     double y;
12     cin >> x >> y;
13     cout << setprecision(17) << expression(x, y) << '\n';
14 }
```

本机 GNU GCC16.2.0 实测：O2、O3 输出 0，Ofast 输出 1。这足以说明 Ofast 并非单纯“更努力优化”；几何谓词和浮点误差分析不能默认沿用原来的语义。

```
1 g++ fast_math.cpp -std=c++23 -O2 -o math_o2
2 g++ fast_math.cpp -std=c++23 -Ofast -o math_fast
3 printf '10000000000000000 1\n' | ./math_o2
4 printf '10000000000000000 1\n' | ./math_fast
```

pragma 的 include 边界也已用同一表达式实测：头文件定义函数 H，include 后定义函数 L，pop 后定义函数 R，三个函数都保留独立调用。外层按 O2 编译。

Ofast pragma 的位置	H / L / R 输出
放在 include 前，L 之后 pop	1 / 1 / 0
放在 include 后，L 之后 pop	0 / 1 / 0

这说明局部选项可以恢复，也说明 include 前的 pragma 会影响头文件定义。不要因为 main 在 pop 之后，就认为它调用的所有函数也恢复成了 O2。

2.3 O2 基线下怎样测

相同输入、校验答案、交错运行候选、多次取中位数；预先约定是否包含初始化、分配和析构。计时区间内不能打印调试信息。必须使用结果，防止编译器删除无用计算。微基准只定位成本，最终仍应跑原题最大数据。

后页实测环境为 Apple M5、macOS arm64、GNU GCC16.2.0、C++23 与 O2，每项 9 个新进程，表中单位为毫秒。计时使用 steady_clock；每次校验整数和。算法主体读取 2048^2 个 int，值为 $1\%1009$ 。同一按行求和循环在 O2、O3、O2 加 unroll 下的中位数见表；差异很小，没有测出稳定的大幅收益。

编译选项	按行求和 (ms)
O2	1.056
O3	1.069
O2 + funroll-loops	1.094

2.4 先减少复制、分配和跨步访问

对比	前者 (ms)	后者 (ms)
逐个扩容 / 一次 reserve	5.152	3.998
按值复制 / const 引用	2.245	1.030
按值复制 / 转移所有权	2.245	1.081
按列访问 / 按行访问	7.308	1.048
每轮新分配 / clear 复用	5.147	4.210
默认 cin / 关闭同步和绑定	307.987	15.015

扩容组从空 vector 开始，计入 reserve、push 和求和；其他数组组先在计时外填好数据。传值组计入参数构造、求和和参数析构；move 后原数组不再使用，引用组仍借用原数组。这些调用的所有权要求不同，先满足接口语义，再考虑快慢。

热点	可直接采用的改动
vector 逐个追加	已知数量时先 reserve(n)，避免多次搬移。别每次追加都 reserve(size()+1)，这会破坏几何扩容。
只读大对象	参数用 const 引用；遍历大元素用 const auto&，避免无意复制。
转交所有权	接口按值接收且原对象之后不用时，传 std::move；const 对象通常仍复制。
多轮临时数组	尺寸相近时复用缓冲区，clear 保留 capacity；峰值很大且不再需要时再考虑释放。
连续矩阵	行主序存储通常让列下标在内层，尽量连续访问；不要在内层反复判断遍历模式。
结构体数组	先看 sizeof 与填充，热点只读少数字段时再试分离字段数组，按实际负载测。

reserve 只增加容量，不产生可下标访问的元素；过度预留会占内存。少量标量按值传递通常更简单，不能把所有参数都改为引用。move 传给 const 引用不会转交所有权，随意给返回值加 move 也可能妨碍复制消除。inline 不强制机器码内联，register 不是通用提速手段。

本次按行/按列都累加相同元素；行主序下前者内层连续，后者步长为 2048 个 int。修改访问顺序必须保留算法依赖，例如背包方向不能为了局部性随意倒置。

2.5 复用缓冲与输入输出

此例比较每轮释放重建与 clear 复用，2048 轮、每轮 4096 个 int，累计校验所有元素。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(int argc, char** argv) {
5     bool fresh = argc > 1 && string(argv[1]) == "fresh";
6     vector<int> a;
7     uint64_t sum = 0;
8     auto start = chrono::steady_clock::now();
9     for (int round = 0; round < 2048; round++) {
10         if (fresh) {
11             vector<int>().swap(a);
12         } else {
13             a.clear();
14         }
15         for (int i = 0; i < 4096; i++) {
16             a.push_back(i ^ round);
17         }
18         sum += accumulate(a.begin(), a.end(), uint64_t(0));
19     }
20     double ms = chrono::duration<double, milli>(chrono::steady_clock::now() - start).count();
21     cout << sum << ' ' << fixed << setprecision(6) << ms << '\n';
22 }

```

输入组从同一磁盘文件读取 100 万个十进制整数（值仍为 $i\%1009$ ），计时包含解析和求和。关闭同步要在 I/O 前做；之后避免混用 stdio 与 istream，交互题应显式刷新。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main(int argc, char** argv) {
5     bool fast = argc > 1 && string(argv[1]) == "fast";
6     if (fast) {
7         ios::sync_with_stdio(false);
8         cin.tie(nullptr);
9     }
10    auto start = chrono::steady_clock::now();
11    uint64_t sum = 0;
12    int x;
13    while (cin >> x) {
14        sum += x;
15    }
16    double ms = chrono::duration<double, milli>(chrono::steady_clock::now() - start).count();
17    cout << sum << ' ' << fixed << setprecision(6) << ms << '\n';
18 }

```

通常用 '\n' 代替逐行 endl；endl 还会刷新。取消 cin 与 cout 绑定后，不能依赖下一次读取帮你刷新提示。

2.6 先检查自动向量化

先写连续数组上的简单循环，以 O2 编译并看报告；循环间依赖、指针可能重叠、复杂分支都可能阻止向量化。

```
1 g++ main.cpp -std=c++23 -O2 -mavx2 \
2   -fopt-info-vec-optimized -o main
3 g++ main.cpp -std=c++23 -O2 -mavx2 \
4   -fopt-info-vec-missed -o main
5 objdump -d -C main > main.asm
```

报告要定位到目标循环；反汇编检查目标函数里的 ymm/zmm 指令，不能只在整份程序里搜到一条就算通过。`__restrict` 承诺指针访问满足不别名要求，只能在实际满足时使用。手写 intrinsics 也需要逐项校验结果，并与编译器生成的版本比较。

2.7 AVX2 类型与常用指令

包含 `<immintrin.h>`。命令行用 `-mavx2`，或用下面的 `push/target/pop` 只包住目标函数。`__m256i` 是 256 位整数向量，可按 8 个 32 位或 4 个 64 位整数操作；`__m256` 为 8 个 float，`__m256d` 为 4 个 double。整数类型本身不区分有符号性，含义由具体指令决定。下表名称均省略前缀 `_mm256_`。

后缀	用途与边界
<code>setzero_si256 / set1_epi32</code>	全零 / 广播一个 32 位值。
<code>loadu_si256 / storeu_si256</code>	读写 32 字节，不要求 32 字节对齐；所有字节仍须有效。
<code>load_si256 / store_si256</code>	地址必须 32 字节对齐；普通 vector 不保证这一点。
<code>add_epi32 / sub_epi32</code>	每个 32 位槽独立加减，低 32 位结果。
<code>mullo_epi32</code>	每槽乘积低 32 位；不是 8 个完整 64 位乘积。
<code>and_si256 / or_si256</code>	逐位与 / 或。
<code>xor_si256 / andnot_si256</code>	异或 / <code>(a)&b</code> ，注意 <code>andnot</code> 参数次序。
<code>cmpeq_epi32 / cmpgt_epi32</code>	相等 / 有符号大于；真槽全 1，假槽全 0。
<code>movemask_epi8</code>	取 32 个字节各自的最高位，组成 32 位掩码。
<code>castsi256_ps + movemask_ps</code>	不转换数值地重解释，再取 8 个 32 位槽的最高位。
<code>permutevar8x32_epi32</code>	按 8 个下标重排整个 256 位向量的 32 位槽。
<code>shuffle_epi8</code>	每个 128 位半区内部按字节重排，不能任意跨半区。

无符号 32 位比较可先把两边异或最高位 `0x80000000u`，再作有符号比较。`cast` 不执行浮点数值转换；下面的 `movemask` 只提取比较结果，不进行浮点运算。

2.8 AVX2 批量加法与条件计数

加法采用 `uint32_t`, 标量与向量都按模 2^{32} 回绕; 不能直接以可能溢出的有符号加法作参考。约定 `n` 非负, 三个数组至少 `n` 个元素, 输出与输入不重叠。每次只读取完整的 8 个元素, 余数走标量。循环写 `i<=n-8`, 避免 `i+8` 在 `n` 接近 `INT_MAX` 时溢出。

```

1 #include <cstdint>
2 #include <immintrin.h>
3
4 #pragma GCC push_options
5 #pragma GCC target("avx2")
6
7 // n >= 0; a, b, out each contain n uint32_t; output does not overlap input.
8 void add_avx2(const uint32_t* a, const uint32_t* b, uint32_t* out, int n) {
9     int i = 0;
10    for (; i <= n - 8; i += 8) {
11        auto x = _mm256_loadu_si256((const __m256i*)(a + i));
12        auto y = _mm256_loadu_si256((const __m256i*)(b + i));
13        auto z = _mm256_add_epi32(x, y);
14        _mm256_storeu_si256((__m256i*)(out + i), z);
15    }
16    for (; i < n; i++) {
17        out[i] = a[i] + b[i];
18    }
19 }
```

计数使用有符号 32 位比较; 提取每槽一个 bit, 再统计 1 的个数。以下函数接在同一个 `target` 范围中, 末尾 `pop` 恢复原来的目标选项。

```

1 int count_less_avx2(const int32_t* a, int n, int32_t bound) {
2     auto limit = _mm256_set1_epi32(bound);
3     int count = 0;
4     int i = 0;
5     for (; i <= n - 8; i += 8) {
6         auto x = _mm256_loadu_si256((const __m256i*)(a + i));
7         auto mask = _mm256_cmpgt_epi32(limit, x);
8         unsigned bits = _mm256_movemask_ps(_mm256_castsi256_ps(mask));
9         count += __builtin_popcount(bits);
10    }
11    for (; i < n; i++) {
12        count += a[i] < bound;
13    }
14    return count;
15 }
16
17 #pragma GCC pop_options
```

2.9 重排与 AVX-512 掩码尾部

逆序 8 个 32 位整数：setr 按书写次序设置槽，槽 0 对应数组第一个元素。输入和输出至少 8 个元素。

```
1 #pragma GCC push_options
2 #pragma GCC target("avx2")
3 void reverse_eight(const int32_t* a, int32_t* out) {
4     auto x = _mm256_loadu_si256((const __m256i*)a);
5     auto order = _mm256_setr_epi32(7, 6, 5, 4, 3, 2, 1, 0);
6     auto y = _mm256_permutevar8x32_epi32(x, order);
7     _mm256_storeu_si256((__m256i*)out, y);
8 }
9 #pragma GCC pop_options
```

AVX-512 的整数向量为 `__m512i`，可容纳 16 个 32 位整数；`__mmask16` 的 bit `j` 控制槽 `j`。下例只需 AVX512F；AVX512BW、DQ、VL 是额外子集，不能由 F 推断它们都可用。最后不足 16 项时，只访问掩码选中的槽，不越过数组尾部。maskz 加载将未选中的槽置零，mask 存储不写未选中的槽。

```
1 #pragma GCC push_options
2 #pragma GCC target("avx512f")
3
4 void add_avx512(const uint32_t* a, const uint32_t* b, uint32_t* out, int n) {
5     int i = 0;
6     for (; i <= n - 16; i += 16) {
7         auto x = _mm512_loadu_si512(a + i);
8         auto y = _mm512_loadu_si512(b + i);
9         _mm512_storeu_si512(out + i, _mm512_add_epi32(x, y));
10    }
11    if (i < n) {
12        __mmask16 mask = (1u << (n - i)) - 1;
13        auto x = _mm512_maskz_loadu_epi32(mask, a + i);
14        auto y = _mm512_maskz_loadu_epi32(mask, b + i);
15        _mm512_mask_storeu_epi32(out + i, mask, _mm512_add_epi32(x, y));
16    }
17 }
18
19 #pragma GCC pop_options
```

此处尾部长度为 1 到 15，因此移位合法；完整 16 项已由主循环处理。loadu 仅免除对齐要求，AVX2 完整加载仍不能跨数组边界；AVX-512 示例则明确使用掩码访存。

2.10 原生 Linux 实测

Ryzen 7 9700X、Linux x86_64、GCC16.2.1，O2 加 `-mavx2`。两个 2^{20} 长度 uint32 数组相加，初始化和逐元素校验在计时外；每个进程执行 100 次，编译器屏障保留每轮写入，9 次交错运行取中位数。普通循环已在目标函数反汇编中确认使用 ymm 向量加法。

实现	编译器循环	手写 AVX2	手写 AVX512F
毫秒	8.538	8.205	7.852

本例三者差距不大；换数据规模、访存方式或 CPU 后重新测量。先检查自动向量化，再对确实有收益的热点保留手写版本。

2.11 编译警告与检查配置

提交版用题目指定标准和优化选项；本地排错单独编译，别拿检查版耗时估算能否通过。常用命令如下，所有源文件同时编译时使用一致的宏和检查选项。

```
1 g++ main.cpp -std=c++23 -O2 -Wall -Wextra -Wshadow -Wconversion -o main
2 g++ main.cpp -std=c++23 -O1 -g -fsanitize=address,undefined \
3   -fno-sanitize-recover=all -fno-omit-frame-pointer -o debug
4 g++ main.cpp -std=c++23 -O1 -g -D_GLIBCXX_DEBUG -o debug_stl
```

选项	用途与易错点
-Wall -Wextra	常见可疑写法；Wall 并不包含所有警告。
-Wshadow	同名局部变量遮蔽外层变量。
-Wconversion	可能改变数值的隐式转换；先核对范围，别为消警告随意强转。
-Wformat	printf/scanf 格式与参数不匹配；Wall 已包含常用检查。
-fsanitize=address	常见堆栈越界、释放后使用；不能指望检出所有未初始化读取或容器逻辑越界。
-fsanitize=undefined	有符号溢出、非法移位等已覆盖的 UB；不是全部 UB 的检测器。
-fno-sanitize-recover=all	检出问题立即停止，不让 UBSan 报错后继续跑出“答案”。
-D_GLIBCXX_ASSERTIONS	libstdc++ 轻量前提检查，如 vector 下标；不跟踪全部迭代器失效。
-D_GLIBCXX_DEBUG	更强容器/迭代器检查；有额外开销，部分容器类型与布局改变。

`_GLIBCXX_DEBUG` 与普通模式不能随意跨编译单元传同一个 STL 容器。ASan 判断可访问内存，vector 的 size 与 capacity 是另一层约束；在已分配容量内访问 size 外元素也非法，但未启用容器注解的 ASan 可能漏掉，要用下标检查或 `at()`。无符号回绕是定义良好的行为，普通 UBSan 不会把它当成有符号溢出。检查只覆盖实际执行路径，仍要用边界数据和对拍。

下例只编译看警告：截断小数、遮蔽变量、printf 类型不匹配、未使用变量。不要执行存在格式类型错误的程序。

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Deliberately suspicious: inspect warnings, do not run this example.
5 int main() {
6     int value = 7;
7     double fraction = 1.5;
8     int truncated = fraction;
9     for (int value = 0; value < 2; value++) {
10         cout << value << '\n';
11     }
12     printf("%lld\n", truncated);
13 }
```

2.12 读懂第一条错误

保存下面故意写错的程序为 `diagnostics.cpp`。每次只执行对应编号；报错后先找第一条诊断和回溯中自己代码的位置，不追最后一次崩溃。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 // Deliberately incorrect: run only to demonstrate the named checker.
5 int main(int argc, char** argv) {
6     if (argc != 2) {
7         return 2;
8     }
9     int mode = stoi(argv[1]);
10    if (mode == 1) {
11        volatile int x = INT_MAX;
12        cout << x + 1 << '\n';
13    } else if (mode == 2) {
14        auto p = make_unique<int[]>(1);
15        volatile int index = 1;
16        cout << p[index] << '\n';
17    } else if (mode == 3) {
18        vector<int> a(1);
19        a.reserve(8);
20        cout << a[3] << '\n';
21    } else if (mode == 4) {
22        vector<int> a{1, 2};
23        auto it = a.begin();
24        a.erase(it);
25        cout << *it << '\n';
26    }
27 }
```

参数	编译时加入	典型报错关键词
1	<code>-fsanitize=undefined</code> <code>-fno-sanitize-recover=all</code>	signed integer overflow
2	<code>-fsanitize=address</code>	heap-buffer-overflow
3	<code>-D_GLIBCXX_ASSERTIONS</code>	下标小于 size 的断言失败
4	<code>-D_GLIBCXX_DEBUG</code>	singular iterator

例如 `g++ diagnostics.cpp -std=c++23 -O1 -g -D_GLIBCXX_DEBUG -o debug`，再执行 `./debug 4`。诊断文本随版本变化，重点看错误类型、访问大小与源代码行。

2.13 值、引用与调用边界

参数 `T` 按值, `const T&` 只读借用, `T&` 允许修改原对象。返回引用必须保证被引用对象仍存活, 不能返回局部变量的引用。结构化绑定 `auto [x,y]` 默认复制, `auto& [x,y]` 引用原元素; 只读遍历常用 `const auto&`。

2.14 `std::move`: 允许移动, 不是立即移动

`std::move(x)` 本身是转换表达式, 使后续重载选择可以使用移动构造或移动赋值; 它自己不搬运数据。对 `const` 对象使用 `move` 不会去掉 `const`, 常见容器因此仍走复制。除有更强保证的特例外, 标准库对象移动后有效但状态未指定: 可销毁、重新赋值, 或调用满足其前提的操作, 不假定内容保持原样或必定为空。

```
1 vector<int> a{1, 2, 3};
2 vector<int> b = std::move(a);
3 a.clear();
4 a.push_back(7);
5 const vector<int> c{4, 5};
6 vector<int> d = std::move(c);
```

上例 `b` 得到原序列, `d` 从 `const c` 复制。调用接受值参数的卷积函数且之后不再需要输入时, 传 `std::move(a)` 可以避免额外副本; 函数参数为 `const` 引用时不会因此取得所有权。返回本地变量通常直接 `return a;`; 多加 `move` 可能失去具名返回值优化机会。

2.15 `vector`、`array`、`deque` 与字符串

`vector` 连续存储, 随机访问常数时间, 尾部追加摊还常数时间。`reserve` 只改容量, 不增加元素; `resize` 才改变 `size`。重分配使原指针、引用和迭代器失效; `erase` 使被删位置及之后失效。不要保存 `vector` 节点引用后再 `push` 新节点; 需要跨扩容引用时保存下标并重新访问。

`array` 的长度为编译期常量; 局部基本类型元素需用 `{}` 初始化。`deque` 支持两端操作且可下标访问, 但整体不连续; 修改后不要盲目保留迭代器。`vector<bool>` 的元素可能是代理对象, 不等同于普通 `bool` 引用。

`string` 适合字符序列; `substr(pos,len)` 第二个参数是长度。`find` 未找到返回 `npos`, 不能直接转 `int` 再判断。`getline` 读整行, 与 `cin>>` 混用时处理遗留换行。字符串反复头部删除可能线性搬移; 需要复杂序列操作见分册 `rope` 与序列树。

2.16 `set`、`map` 与哈希表

`set/map` 去重, `multiset/multimap` 允许等价键。比较器必须是严格弱序; 不能用 `less_equal` 假装多重集合, 也不直接用不传递的 `epsilon` 比较做排序或树比较。`map` 的下标访问会插入默认值, 只查询用 `find`。

浮点键若含 `NaN`, 普通小于比较也不再满足严格弱序: `NaN` 与不同有限数都可能互不小于, 但这些有限数之间仍有大小关系。几何排序前应遵守有限输入约定, 不能把 `NaN` 或 `epsilon` 当成普通相等键。

`multiset` 的 `erase(key)` 会删掉全部等价元素; 只删一个时先 `find`, 再 `erase` 该迭代器。树容器的 `lower_bound` 用成员函数, 通用算法在非随机访问迭代器上可能线性走步。关联容器的键不可原地修改为破坏排序的值。

`unordered_map` 的普通查改平均常数时间, 最坏可线性; `rehash` 会使迭代器失效。需要容量控制时看其 `reserve/max_load_factor` 接口, 不把这些名字照搬到 `PBDS`。GNU 的 `gp/cc` 哈希表、选型和范围见本册末的分册索引。

2.17 queue、stack 与优先队列

queue 先入先出，stack 后入先出；取 front/top 和 pop 前检查非空。标准 priority_queue 默认大根堆，pop 不返回值，先复制 top 再 pop。它没有按句柄修改和合并接口。

```
1 priority_queue<int, vector<int>, greater<int>> q;
2 q.push(7);
3 q.push(2);
4 int x = q.top();
5 q.pop();
```

需要修改已入堆的键、删除指定元素或合并时，使用数据结构册的 PBDS 配对堆；不要混用两种 priority_queue 的接口或句柄保证。

2.18 排序、查找与区间算法

算法区间通常为半开区间。sort 要求严格弱序，stable_sort 保留等价元素的原顺序；nth_element 只保证第 k 位及两侧分区，平均线性，不等于完整排序。lower/upper_bound 要求对应有序或分区前提，并使用一致比较规则。

```
1 sort(a.begin(), a.end());
2 a.erase(unique(a.begin(), a.end()), a.end());
3 auto it = lower_bound(a.begin(), a.end(), 7);
4 if (it != a.end()) {
5     cout << *it << '\n';
6 }
```

unique 只合并相邻等价值，返回逻辑终点；它不改容器 size。remove/remove_if 同理，配合 erase；C++20 可用适用容器的 erase/erase_if。reverse、rotate、merge、copy 的目标区间必须有效，不能把未扩容的 vector.begin() 当作足够大的输出区间。

2.19 pair、tuple、optional 与回调

pair/tuple 默认按字典序比较，适合把值与唯一编号组合。optional 区分无解与合法零值，先判断再解引用。std::function 可保存不同可调用对象，但有类型擦除开销；泛型 lambda 或模板回调更容易内联。递归 lambda 捕获的引用不能活得比被引用对象更久。

tie(a,b) 创建引用元组，保存后仍引用原变量，不是值快照。次小生成树用它比较权值与编号；模整数扩展欧几里得用右侧的值 pair 先保存旧值，再赋给左侧引用。不能把 tie(a,b)=tie(b,a) 当作可靠的交换：右侧也在引用会被改写的变量。

```
1 long long a = 17, b = 5;
2 long long q = a / b;
3 tie(a, b) = pair{b, a - q * b};
4 assert(a == 5 && b == 2);
```

普通数值变量要保存快照可用 pair{a,b} 或 make_tuple(a,b)，交换则直接 swap。

2.20 数值累加与前缀和

`accumulate` 的初值决定累加类型，64 位求和用 `0LL`；把返回结果装进 `long long` 不会改变中间累加类型。乘积也须先提升，例如 `1LL*a*b`，但仍应保证宽类型可表示结果。

`partial_sum` 的累加器类型取自输入迭代器的值类型，不由输出容器决定。SAM 的长度桶前缀计数用 `int`，前提是总状态数可表示；更大的前缀和应先将输入转为 `long long`，或用显式的宽类型循环。只把输出改为 `vector<long long>` 不能避免 `int` 累加溢出。允许输出起点等于输入起点：

```
1 vector<int> a{1000000000, 1000000000, 1000000000};
2 vector<long long> prefix(a.begin(), a.end());
3 partial_sum(prefix.begin(), prefix.end(), prefix.begin());
4 assert(prefix.back() == 3000000000LL);
```

空区间不写输出；非空时输出区间必须足够大。`iota` 适合连续编号。`gcd/lcm` 仍有可表示范围前提，最小负数的绝对值应先转宽或转无符号幅值。浮点整数精确范围和 `long double` 有效位数用 `numeric_limits` 查询；几何数值条件另看几何册。

2.21 complex 与 FFT

FFT 使用 `complex<long double>`；`C(re,im)` 分别设置实部、虚部，`real()/imag()` 读取分量，`conj(z)` 取共轭。`abs(z)` 是模长，`norm(z)` 是模长的平方；尤其不要与本库 `RealPlane::norm` 的长度含义混淆。逆变换使用共轭单位根，并按模板约定除以长度。

```
1 using C = complex<long double>;
2 C z(3, 4);
3 assert(z.real() == 3 && z.imag() == 4);
4 assert(conj(z) == C(3, -4));
5 assert(abs(z) == 5 && norm(z) == 25);
```

复数乘法仍是浮点运算，`norm` 也可能溢出，不能因 API 名称而放宽 FFT 的输入范围与舍入误差界。精确整数卷积是否适用浮点版本，应看数学册说明。

2.22 C++20 位操作与零边界

`bit_width`、`popcount`、`countl_zero`、`countr_zero` 接受无符号整数类型。`bit_width(x)` 表示 `x` 所需的二进制位数，零返回 0；`popcount` 返回 1 的个数。两个 `count-zero` 函数对零都返回该类型的位宽，不是 0。不要把 `countl_zero` 的结果直接当作最高位下标。

```
1 static_assert(bit_width(0u) == 0);
2 static_assert(bit_width(8u) == 4);
3 static_assert(popcount(10u) == 2);
4 static_assert(countr_zero(0u) == numeric_limits<unsigned>::digits);
```

划分树、WaveletMatrix 用 `bit_width((unsigned)max(1,n)-1)` 这类表达式计算层数；其中 `n` 分别取数组长度与去重后的值域大小。先保护零再减一，避免无符号回绕；各模板是否另加一层按接口说明。GCC 的 `__builtin_ctz(0)` 与 `__builtin_clz(0)` 均不能调用，处理 64 位输入还须选对应 `ll` 版本。

`1ULL<<k` 须满足 `k` 非负且小于 `unsigned long long` 的位宽。WaveletMatrix 按 64 位块存储，用 `k=i%64` 得到 0 到 63；`k=0` 时 `(1ULL<<k)-1` 为零掩码，不能用移位 64 构造整块掩码。`bitset` 的大小固定，批量位运算不是与位数无关的常数操作。

2.23 计时与卡时

`steady_clock` 单调，适合测经过时间；`system_clock` 是日历时间，可能被校时。`high_resolution_clock` 可能只是前两者之一的别名，不能据名字假定单调。`duration_cast<milliseconds>` 截掉不足 1 毫秒的部分；`duration<double>` 得到以秒为单位的浮点数。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     using Clock = chrono::steady_clock;
6     auto start = Clock::now();
7     auto deadline = start + chrono::milliseconds(20);
8     mt19937_64 rng(start.time_since_epoch().count());
9     uint64_t checksum = 0;
10    do {
11        for (int i = 0; i < 1024; i++) {
12            checksum ^= rng();
13        }
14    } while (Clock::now() < deadline);
15    auto elapsed = Clock::now() - start;
16    auto ms = chrono::duration_cast<chrono::milliseconds>(elapsed).count();
17    double seconds = chrono::duration<double>(elapsed).count();
18    cout << checksum << '\n';
19    cerr << ms << " ms, " << seconds << " s\n";
20 }

```

上例每 1024 次操作检查一次时间，减少读钟开销；卡时可能超出一个批次，还要为输出留余量。时间种子可改变每次运行的序列；对拍改成固定种子，并保存失败输入。随机搜索要随时保存合法的当前最优解，不能等超时后才构造答案。

2.24 随机引擎、分布与抽样

`mt19937` 产生 32 位值，`mt19937_64` 产生 64 位值。不要用 `rng()%n` 代替均匀分布：除非区间大小整除引擎值域大小，否则有偏差。`rand()` 的值域和质量依实现；`random_shuffle` 已在 C++17 移除，改用 `shuffle`。

调用	结果与条件
<code>uniform_int_distribution<long long>(l,r)</code>	整数闭区间 $[l,r]$ ，要求 $l \leq r$ 。
<code>uniform_real_distribution<double>(l,r)</code>	目标区间 $[l,r]$ ；用有限小范围且 $l < r$ ，注意浮点舍入。
<code>bernoulli_distribution(p)</code>	概率 p 为 true， $0 \leq p \leq 1$ 。
<code>normal_distribution<double>(mu,sigma)</code>	正态分布；标准差 $\sigma > 0$ ，结果不限区间。
<code>discrete_distribution<int>{1,2,7}</code>	返回下标 0、1、2，概率比 1:2:7；权非负且总和正。
<code>shuffle(first,last,rng)</code>	原地打乱，保留元素和重复次数。
<code>sample(first,last,out,k,rng)</code>	无放回抽取至多 k 个位置；值不一定互异。

分布对象可能缓存结果（例如正态分布）。重设引擎种子后，要复现整条调用序列，还须对已用分布调用 `reset()` 或重新构造。同一工具链中固定种子与调用顺序可复现；不同标准库的分布和洗牌结果不保证逐项相同。

2.25 对拍用造数据函数

以下代码保存为 random_tools.hpp；点编号为 $[0, n)$ ，长度非负。互异值先生成全集再抽取，适合可枚举的小值域；随机字符串的字母表非空。树采用逐点连接旧点再随机重标号，并非所有标号树等概率。简单无向图先枚举全部边再抽取，适合小数据，不保证连通。对拍还应专门构造链、星、完全图、空图、重复权等边界，别只依赖一种随机分布。

```

1 #include <bits/stdc++.h>
2 #include <cassert>
3 using namespace std;
4
5 mt19937_64 rng(712367821);
6
7 long long random_int(long long l, long long r) {
8     return uniform_int_distribution<long long>(l, r)(rng);
9 }
10
11 double random_real(double l, double r) {
12     return uniform_real_distribution<double>(l, r)(rng);
13 }
14
15 vector<int> permutation(int n) {
16     vector<int> p(n);
17     iota(p.begin(), p.end(), 0);
18     shuffle(p.begin(), p.end(), rng);
19     return p;
20 }
21
22 // Choose k distinct values from [0, n), using O(n) space.
23 vector<int> distinct_values(int n, int k) {
24     assert(0 <= k && k <= n);
25     auto p = permutation(n);
26     p.resize(k);
27     return p;
28 }
29
30 string random_string(int n, const string& alphabet) {
31     assert(!alphabet.empty());
32     string s(n, ' ');
33     for (char& c : s) {
34         c = alphabet[random_int(0, (int)alphabet.size() - 1)];
35     }
36     return s;
37 }
38
39 // Random recursive tree, then relabel; not uniform over all trees.
40 vector<pair<int, int>> random_tree(int n) {
41     auto p = permutation(n);
42     vector<pair<int, int>> edges;
43     for (int i = 1; i < n; i++) {
44         edges.emplace_back(p[i], p[random_int(0, i - 1)]);
45     }
46     shuffle(edges.begin(), edges.end(), rng);
47     return edges;
48 }

```

```

50 // Small undirected simple graphs;  $O(n^2)$  time and space.
51 vector<pair<int, int>> random_graph(int n, int m) {
52     assert(0 <= m && m <= 1LL * n * (n - 1) / 2);
53     vector<pair<int, int>> edges;
54     for (int u = 0; u < n; u++) {
55         for (int v = u + 1; v < n; v++) {
56             edges.emplace_back(u, v);
57         }
58     }
59     shuffle(edges.begin(), edges.end(), rng);
60     edges.resize(m);
61     return edges;
62 }

```

2.26 生成器入口

下面入口与 random_tools.hpp 放在同一目录，编译后以命令行参数传种子。演示输出先打印各类随机值，最后打印图；对拍时只保留题目需要的输出格式。

```

1 #include "random_tools.hpp"
2
3 int main(int argc, char** argv) {
4     if (argc > 1) {
5         rng.seed(stoull(argv[1]));
6     }
7     bernoulli_distribution coin(0.3);
8     normal_distribution<double> normal(0.0, 1.0);
9     discrete_distribution<int> weighted{1, 2, 7};
10    cout << coin(rng) << ' ' << normal(rng) << ' ' << weighted(rng) << '\n';
11    auto p = permutation(10);
12    vector<int> chosen;
13    sample(p.begin(), p.end(), back_inserter(chosen), 3, rng);
14    for (int x : chosen) {
15        cout << x << ' ';
16    }
17    cout << '\n';
18    cout << random_int(-10, 10) << ' ' << random_real(0, 1) << '\n';
19    cout << random_string(12, "abc") << '\n';
20    auto edges = random_graph(6, 8);
21    cout << 6 << ' ' << edges.size() << '\n';
22    for (auto [u, v] : edges) {
23        cout << u << ' ' << v << '\n';
24    }
25 }

```

2.27 整数键防卡哈希

整数键可用时间种子配合 `splitmix64` 混合，降低针对固定哈希构造冲突的风险。它不保证最坏时间，也不适合密码用途。种子一经初始化，在容器存活期间保持不变；不要每次调用哈希时重新取时间。PBDS 没有与 `unordered_map` 相同的 `reserve` 接口。

```

1 #include <bits/stdc++.h>
2 #include <cassert>
3 #include <ext/pb_ds/assoc_container.hpp>
4 using namespace std;
5
6 struct Hash {
7     static uint64_t mix(uint64_t x) {
8         x += 0x9e3779b97f4a7c15ULL;
9         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
10        x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
11        return x ^ (x >> 31);
12    }
13
14    size_t operator()(uint64_t x) const {
15        static const uint64_t seed =
16            chrono::steady_clock::now().time_since_epoch().count();
17        return mix(x + seed);
18    }
19 };
20
21 int main() {
22     unordered_map<long long, int, Hash> a;
23     __gnu_pbds::gp_hash_table<long long, int, Hash> b;
24     a.reserve(1000);
25     for (long long x : {-1LL, 0LL, LLONG_MIN, LLONG_MAX}) {
26         a[x]++;
27         b[x]++;
28         assert(a.at(x) == b.find(x)->second);
29     }
30     cout << a.size() << ' ' << b.size() << '\n';
31 }

```

2.28 GNU 扩展与查询范围

PBDS 有序树、gp/cc 哈希表、配对堆以及 `__gnu_cxx::rope` 在所属知识点详细介绍，本册提供集中页码跳转。`rope` 不叫 `std::rope`；复制版本的内存成本和已知失败用法不因它是库容器而消失。

2.29 Boost 任意精度整数

`boost::multiprecision::cpp_int` 是有符号任意精度整数，须显式包含 `boost/multiprecision/cpp_int.hpp`；它不属于 ISO C++ 标准库或 GNU 标准头文件，赛场必须提供 Boost。可使用十进制流输入输出，整数除法仍是截断除法，不会自动变成有理数，除数不能为零。Bareiss 消元依靠数学上的整除性质保存精确整数，矩阵树用它计算不取模答案；目录末有对应分册跳转。数值越大，运算和存储越贵，不应把任意精度操作视为机器字常数时间。Boost 使用表达式模板，需跨语句保存计算结果时写明确的 `cpp_int` 类型，避免用 `auto` 保存依赖随后修改变量或临时量的表达式。

2.30 `__int128` 的十进制读写

有符号范围 $[-2^{127}, 2^{127} - 1]$ ，无符号范围 $[0, 2^{128} - 1]$ 。先转换再乘，如 `i128(a)*b`；有符号溢出仍是 UB。最小负数不能直接取负，也不能先读到 `long long` 再转换。下面函数按字符串解析，拒绝非法或超范围输入；失败时保留原变量。输出先取无符号幅值，正确处理最小负数与 0。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using i128 = __int128;
4 using u128 = unsigned __int128;
5
6 // Invalid or out-of-range token: return false, leave x unchanged.
7 bool parse128(const string& s, i128& x) {
8     if (s.empty()) {
9         return false;
10    }
11    bool neg = s[0] == '-';
12    size_t pos = neg || s[0] == '+';
13    if (pos == s.size()) {
14        return false;
15    }
16    u128 limit = (u128(1) << 127) - !neg;
17    u128 value = 0;
18    for (; pos < s.size(); pos++) {
19        if (s[pos] < '0' || s[pos] > '9') {
20            return false;
21        }
22        unsigned digit = s[pos] - '0';
23        if (value > (limit - digit) / 10) {
24            return false;
25        }
26        value = value * 10 + digit;
27    }
28    x = neg && value ? -i128(value - 1) - 1 : i128(value);
29    return true;
30 }
31
32 string str128(i128 x) {
33     bool neg = x < 0;
34     u128 value = neg ? u128(0) - u128(x) : u128(x);
35     string s;
36     do {
37         s.push_back('0' + value % 10);
38         value /= 10;
39     } while (value);
40     if (neg) {
41         s.push_back('-');
42     }
43     reverse(s.begin(), s.end());
44     return s;
45 }

```

2.31 strict 与 GNU 模式的差异

前页函数后接以下入口即可读到 EOF；普通 cin/cout 没有直接的 `__int128` 数值重载。

```

47 int main() {
48     string s;
49     while (cin >> s) {
50         i128 x = 0;
51         if (parse128(s, x)) {
52             cout << str128(x) << '\n';
53         } else {
54             cout << "invalid\n";
55         }
56     }
57 }
```

GCC 在支持 128 位整数的目标上，`-std=c++20` 也接受 `__int128` 扩展，但它不是 ISO 标准类型。

加入 `-pedantic-errors` 会拒绝此扩展写法。`-std=gnu++20` 关闭 strict 模式，部分版本的标准库还会改变对扩展类型的识别，不能因为能声明变量就假定所有模板都接受它。

```

1  #include <iostream>
2  #include <limits>
3  #include <type_traits>
4  using namespace std;
5
6  int main() {
7      cout << __cplusplus << '\n';
8      #ifdef __STRICT_ANSI__
9          cout << 1 << '\n';
10     #else
11         cout << 0 << '\n';
12     #endif
13     cout << sizeof(__int128) << '\n';
14     cout << is_integral_v<__int128> << '\n';
15     cout << numeric_limits<__int128>::digits << '\n';
16 }
```

本机 GNU GCC16、libstdc++ 实测：以上探针在两种模式分别输出 202002 / 1 / 16 / 1 / 127 与 202002 / 0 / 16 / 1 / 127。本版本两种模式的 `is_integral_v<__int128>` 均为 true；区别是 strict 宏。标准库约束与可用接口还取决于版本。热身时用比赛的真实编译命令跑探针；不要用该 trait 在 strict 模式下否定编译器的算术支持。GNU 头文件、PBDS 和 bitset 扩展同样不是 ISO 接口；算法分册页码见本册末索引。

2.32 GNU 位函数与 bitset 遍历

写法	结果或条件
<code>__builtin_popcountll(x)</code>	unsigned long long 中 1 的个数，允许 0。
<code>__builtin_ctzll(x)</code>	从低位数连续 0; $x \neq 0$ 。
<code>__builtin_clzll(x)</code>	从高位数连续 0; $x \neq 0$, 64 位时最高位下标为 63 减该值。
<code>x &= x-1</code>	对无符号非零数删去最低位的 1。
<code>b._Find_first()</code>	最低置位位置，无置位返回 <code>b.size()</code> 。
<code>b._Find_next(i)</code>	严格大于 i 的下一置位，无则返回 <code>size</code> 。

无 ll 的版本对应 unsigned int; 64 位值误传会被截断。bitset 两个查找函数是 libstdc++ 扩展，可跨多个机器字扫描，并非总是常数时间。

以下例子同时覆盖第 0、64、129 位以及空集合；遍历条件必须排除 size 哨兵。

```

1 #include <bits/stdc++.h>
2 #include <cassert>
3 using namespace std;
4
5 vector<int> positions(uint64_t x) {
6     vector<int> p;
7     while (x) {
8         p.push_back(__builtin_ctzll(x));
9         x &= x - 1;
10    }
11    return p;
12 }
13
14 int main() {
15     uint64_t x = (1ULL << 63) | 5;
16     assert(__builtin_popcountll(x) == 3);
17     assert(63 - __builtin_clzll(x) == 63);
18     assert(positions(x) == vector<int>({0, 2, 63}));
19     assert(positions(0).empty());
20     bitset<130> b;
21     b.set(0);
22     b.set(64);
23     b.set(129);
24     for (size_t i = b._Find_first(); i < b.size(); i = b._Find_next(i)) {
25         cout << i << '\n';
26     }
27     b.reset();
28     assert(b._Find_first() == b.size());
29     return 0;
30 }
```

第三章 Shell 脚本与常用命令

3.1 编译并运行

保存为 compile.sh, 运行 `bash compile.sh main`; 默认调试编译后运行。加 `-O2` 使用优化配置; `-o solution` 指定输出且只编译。输入输出可在调用后接 `< input.txt > actual.txt`。

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3 file=
4 out=
5 run=1
6 flags=(-Wall -Wextra -Wno-sign-compare -std=c++23)
7 mode=(-O1 -g -fsanitize=address,undefined -fno-sanitize-recover=all)
8 while (($#)); do
9     case "$1" in
10         -O2)
11             mode=(-O2)
12             ;;
13         -o)
14             [[ $# -ge 2 && $2 != -* ]] || exit 2
15             out=$2
16             run=0
17             shift
18             ;;
19         -*)
20             echo "unknown option: $1" >&2
21             exit 2
22             ;;
23         *)
24             [[ -z $file ]] || exit 2
25             file=$1
26             ;;
27     esac
28     shift
29 done
30 [[ -n $file ]] || exit 2
31 file="${file%.cpp}.cpp"
32 [[ -f $file ]] || exit 2
33 out=${out:-${file%.cpp}}
34 [[ $out == /* ]] || out="./$out"
35 [[ ! -L $out && ! $file -ef $out ]] || exit 2
36 [[ ! -e $out || ( -f $out && -x $out ) ]] || exit 2
37 g++ "$file" -o "$out" "${flags[@]}" "${mode[@]}"
38 echo "compiled: $out" >&2
39 if ((run)); then
40     "$out"
41 fi
```

只支持单源文件, 文件名不以减号开头。路径含空格时加引号。拒绝覆盖源码、普通文本、目录和链接; 已有普通可执行文件可覆盖。编译失败不运行旧文件; 程序运行失败保留非零退出状态。

3.2 对拍与保存反例

当前目录准备可执行的 gen、brute、main：生成器接收整数种子，暴力和待测从标准输入读取。执行 `bash stress.sh 1000`。

```
1 #!/usr/bin/env bash
2 set -euo pipefail
3 count=${1:-1000}
4 [[ $count =~ ^[1-9][0-9]*$ ]] || exit 2
5 dir=$(mktemp -d ./stress.XXXXXX)
6 echo "saved in $dir" >&2
7 trap 'echo "stopped at seed $seed; files: $dir" >&2' ERR
8 for ((seed=1; seed<=count; seed++)); do
9     echo "$seed" > "$dir/seed"
10    for name in input expected actual diff gen.err brute.err main.err; do
11        : > "$dir/$name"
12    done
13    timeout -k 1s 2s ./gen "$seed" > "$dir/input" 2> "$dir/gen.err"
14    timeout -k 1s 2s ./brute < "$dir/input" > "$dir/expected" 2> "$dir/brute.err"
15    timeout -k 1s 2s ./main < "$dir/input" > "$dir/actual" 2> "$dir/main.err"
16    diff -u "$dir/expected" "$dir/actual" > "$dir/diff"
17 done
18 echo "passed $count seeds; last case: $dir"
```

每个程序分别限时 2 秒，TERM 后 1 秒仍未退出则 KILL。timeout 返回 124 通常为超时，137 可能为 KILL；检查对应 err 文件。diff 返回 1 表示答案不同，其他错误也停下。浮点容差题、输出方案题应换为专用 checker。每次创建独立目录，每轮清空输出槽；失败保存首个反例，成功保留末组。用保存的 input 直接重跑，不能只凭种子猜测输入。

3.3 完整造数与暴力示例

示例任务为非空最大子段和。保存下列生成器为 `gen.py`，暴力为 `brute.py`；待测 `main` 使用自己的算法。复制为 `gen`、`brute` 后执行 `chmod +x gen brute`，用前页脚本对拍。

```
1 #!/usr/bin/env python3
2 import random
3 import sys
4
5 rng = random.Random(int(sys.argv[1]))
6 n = rng.randint(1, 12)
7 a = [rng.randint(-20, 20) for _ in range(n)]
8 print(n)
9 print(*a)
```

```
1 #!/usr/bin/env python3
2 import sys
3
4 data = list(map(int, sys.stdin.buffer.read().split()))
5 n = data[0]
6 a = data[1:]
7 assert len(a) == n
8 # Nonempty maximum subarray sum.
9 print(max(sum(a[l:r]) for l in range(n) for r in range(l + 1, n + 1)))
```

写法	用途与坑点
"\$1"、\$#、shift	首参数、参数个数、移除首参数；路径变量必须双引号。
flags=(...)	数组；"\${flags[@]}" 逐项传参，不能把选项全塞入一个字符串。
[[...]], ((...))	字符串/文件判断、算术条件。算术结果 0 的退出状态是 1。
<、>、2>	输入、覆盖输出、错误输出；>> 追加，别把同一文件同时当输入输出。
set -euo pipefail	普通命令失败、未设置变量、管道失败尽早停止；if 条件和 &&/ 链等有例外。
head -n 20 file	看前 20 行；wc -l file 数换行，sort -n file 数值排序。
diff -u a b	对比内容；cat run.err 检查错误。编译成功不等于运行成功。

第四章 赛场环境与调试

4.1 提交前的接口核对

先对照题面确认下标、闭区间或半开区间、空输入、重边、自环和数值范围。返回值为 0 可能是合法答案，不能一律当作失败。算法模板的数学前提与输出方案约定优先于类型名的直觉。

本库 C++20 模板通常需要 `bits/stdc++.h`、`cassert` 和 `std` 命名空间。GNU 扩展不是标准 C++；若评测环境没有 `libstdc++`，不要假定 `PBDS`、`rope`、`bits/stdc++.h` 可用。整数乘积先转宽类型再乘，不能乘完才强转。断言用于检查调用前提，不能把断言本身当作正确性证明。

4.2 对拍与性能检查

小数据参考应尽量用枚举、直接模拟或不同数学公式。固定随机种子并保存失败输入；先检查样例、空边界、极值、重复值，再测试最大规模。消毒器检查越界与未定义行为，优化版单独测时间。相同机器上的本地耗时也不能直接换算为 OJ 排名。

递归 DFS 保持递归。本机默认线程栈较小时调整本机测试环境，不因此改成显式模拟 DFS。算法的栈深度与题目内存约束仍需匹配。

4.3 分册跳转索引

请将本册与其他 PDF 放在同一目录。下表页码是分册印刷页码；链接直接指向条目，部分浏览器内置 PDF 阅读器可能不支持跨文件跳转。

内容	所属分册及页码
PBDS 配对堆：修改、合并与句柄	数据结构，第 16 页
PBDS 有序树：排名与第 k 小	数据结构，第 91 页
GNU rope：序列与版本共享	数据结构，第 89 页
gp/cc 哈希表：整数键映射	数据结构，第 1 页
优先队列的最短路应用	图论，第 74 页
vector 与下标：树状数组	数据结构，第 35 页
扩容与下标：可持久化数组	数据结构，第 150 页
比较次序：后缀数组	字符串，第 29 页
数值精度约定：浮点几何	计算几何，第 1 页
移动大数组：NTT 卷积	数学，第 86 页
complex 的平方模与共轭：FFT	数学，第 78 页
位宽与压位计数：小波矩阵	数据结构，第 84 页
tie 与值比较：次小生成树	图论，第 118 页
精确整数行列式：Boost cpp_int	数学，第 171 页
不取模的带权生成树计数	图论，第 265 页
PBDS 句柄删除：双端优先队列	数据结构，第 17 页
PBDS modify 降键：最短路与路径	数据结构，第 18 页
move 数组：批量逆元与动态模数	数学，第 8 页

用途索引

accumulate, 16
array, 14
ASan, 12, 24
AVX-512, 11
AVX2, 9
bit-width, 16
bitset, 16, 23
Boost, 20
builtin, 23
chrono, 17
complex, 16
conj, 16
cpp-int, 20
deque, 14
diff, 25
function, 15
GNU 扩展, 22
heapq, 2
int128, 21
intrinsics, 9
itertools, 2
lower-bound, 15
map, 14
multiset, 14
nth-element, 15
numeric, 16
O2, 5
O3, 5
Ofast, 5
optional, 15
pair, 15
partial-sum, 16
PBDS, 20
popcount, 16
pragma, 5
priority-queue, 15
Python, 1
Python 输入输出, 4
queue, 15
random, 17
reference, 14
reserve, 7
rope, 20
sample, 17
set, 14
shuffle, 17
SIMD, 9
SIMD 尾部, 10
SIMD 重排, 11
sort, 15
splitmix64, 20
stack, 15
std::move, 14
string, 14
timeout, 25
tuple, 15
UBSan, 12, 24
unique, 15
unordered-map, 14
vector, 14
低位 1, 23
内存布局, 7
列表别名, 4
卡常, 7
卡时, 17
大整数, 1
对拍, 25, 27
性能测试, 6
打表, 3
提交检查, 27
整数转字符串, 4
暴力, 26
最小负数, 21
模逆, 1
浮点重排, 6
缓冲区, 8
编译脚本, 24
自动向量化, 9
警告, 12
计时, 17
记忆化, 2
语言标准, 22
越界, 12
输入输出优化, 8
运行时排错, 13
迭代器失效, 13
造数据, 3, 18
防卡哈希, 20
随机图, 18
随机树, 18